# Ders 7 Genişletilmiş Notu: Proje Export/Import, Kopya Yapıcı, Senaryodan Sınıf Çıkarma ve Nesneler Arası İlişkiler: Association ve Dependency

## Ders 7 Genişletilmiş Notu: Proje Export/Import, Kopya Yapıcı, Senaryodan Sınıf Çıkarma ve Nesneler Arası İlişkiler: Association ve Dependency

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

### İçindekiler


- 1. Proje Export ve Import
- 2. Bu Dersin Diğer Derslerle Bağlantısı
- 3. Kopya Yapıcı (Copy Constructor)
- 4. C++'ta Kopya Yapıcı ve Yıkıcı
- 5. Senaryodan Sınıf ve Özellik Çıkarma: Üniversite Bilgi Sistemi
- 6. Adres Defteri Örneği ve Yazılım Tasarımının Aşamaları
- 7. Nesneler Arası İlişkiler
- 8. Association
- 9. Ok Yönü ve Kardinalite
- 10. Az Ayrıntılı ve Çok Ayrıntılı Sınıf Diyagramı
- 11. Gizli Bilgi (Hidden Information)
- 12. Dependency
- 13. Tek Yönlü Association: Person ve Car
- 14. `null` Kontrolü ve Sağlam Kod
- 15. Başlatma Testi, Referans ve Heap
- 16. Sequence Diagram ile İzleme
- 17. Diyagramlardaki Gizli İlişkiler
- 18. Çift Yönlü Association
- 19. Veritabanı Tarafından Bakış


## 1. Proje Export ve Import

### 1.1 Bir Java Projesi Neden Tek Dosya Değildir?

Hoca derse, bir projeyi başka bir yere taşımanın nasıl yapıldığını anlatarak başladı. NetBeans, Eclipse ya da IntelliJ kullanın, yazdığınız kod bir proje biçiminde çalışır. C'de alışık olduğumuz gibi tek bir kod dosyası yoktur. Projenin yapılandırma (configuration) dosyaları, kütüphaneleri (library) ve JDK ile ilgili birçok parçası vardır. Bu nedenle kodu başka bir yere taşımak istediğinizde **export** ve **import** işlemlerini bilmeniz gerekir. Hocanın örneğiyle, teslim ettiğiniz bir proje sizin tarafınızdan export, onu alan taraftan import edilir.

Bu işlemler yapılmazsa elle de çözüm vardır: Yeni bir proje oluşturup Java dosyalarını kopyalarsınız. Kütüphane kullanmadığımız sürece bu pek sorun çıkarmaz. Fakat paket adlarını düzeltmeniz gerekebilir. Hoca burada önce "namespace" dedi, sonra düzeltti: C++'taki namespace'in Java'daki karşılığı pakettir (package), aynı şeyin farklı adlandırılmasıdır.

### 1.2 NetBeans'te Export ve Import

Hoca işlemi NetBeans üzerinde adım adım gösterdi. Export şöyle yapılır:

1. `File > Export Project > To ZIP` seçilir.
2. Açılan pencerede projeler listelenir. Hangi proje isteniyorsa o seçilir ya da herhangi bir dizinden proje gösterilir.
3. `Browse` ile hedef yer seçilir ve dosya adı yazılır.
4. `Export` denir.

Hoca ilk denemede dosya adını `.zip` uzantısı olmadan yazdı ve bu yüzden dosya ZIP olarak tanınmadı. Dersin sonunda aynı işlemi `.zip` uzantısıyla tekrarladı ve masaüstünde ZIP dosyası sorunsuz oluştu. Yani hedef dosya adına uzantıyı siz yazmalısınız.

Import ise şöyle yapılır:

1. `File > Import Project > From ZIP` seçilir.
2. `Browse` ile ZIP dosyası gösterilir.
3. Projenin açılacağı klasör seçilir. Varsayılan yer NetBeans projeleri klasörüdür.
4. Aynı adlı proje varsa IDE uyarır. Hocanın denemesinde `Hafta4` zaten vardı. Bu durumda `Change import folder` ile başka bir klasör seçilir.
5. `Import` denir. İşlem arka planda yapılır ve proje listede görünür.

Benzer işlemler NetBeans ile Eclipse arasında da yapılabilir. Hoca NetBeans'in Eclipse projelerini de import edebildiğini gösterdi.

### 1.3 Eclipse'te Export ve Import

Hoca, asistanın hazırladığı Eclipse dokümanı üzerinden işlemi özetledi. Import menüsünde dosyanın yolu gösterilir, dosyanın içindeki projeler listelenir ve `Finish` denir. Export ederken de export seçilir, arşiv dosyası (ZIP) biçimi işaretlenir ve aktarılacak dosyalar seçilir. Hocaya göre en kolay biçim ZIP'tir. Seçenekler IDE'den IDE'ye değişir, ama işlem zor değildir.

### 1.4 Karakter Kodlaması ve Projenin Diskteki Yeri

Gönderilen kodda bazen karakter kodlamasından (encoding) kaynaklı sorun çıkar. Bu yüzden aktarımda **UTF-8** seçilmelidir. Export penceresinde bunun için seçenekler de vardır. Hoca dersin ilerleyen kısmında hazır kodu NetBeans'e kopyalarken de bir UTF-8 hatasıyla karşılaştı.

Projenin diskte nerede olduğunu bilmeyenler için yol şudur: Projeye sağ tıklayıp `Properties` dersiniz. Eclipse'te proje genellikle workspace klasörünün altındadır. NetBeans'te ise Windows kullanılıyorsa Belgeler (Documents) klasöründeki NetBeans projeleri klasörünün altındadır.

### 1.5 `build` ve `src` Klasörleri

Proje klasöründe kaynak kodun yanında onun derlenmiş aynası da bulunur. Hoca `Hafta2` projesini açtı. IDE'de kaynak paketler (source packages), test paketleri ve kütüphaneler görünüyordu. Klasöre doğrudan girildiğinde ise `build` altında `Kitap.class`, `Kutuphane.class`, `Test.class` ve `Yazar.class` dosyaları vardı.

Kod gönderirken hata yapanların çoğu bu `.class` dosyalarını gönderir. Bu dosyalar derlenmiş bayt kodudur (bytecode). Hoca bunların decompiler ile açılabildiğini, ama tam olarak geri çevrilemediğini söyledi. Kaynak kodu görmek gerektiğinde işe yaramazlar. Elle kopyalamak istiyorsanız `src` klasörüne girmelisiniz. Oradaki `Hafta2` klasörü, IDE'de gördüğünüz `Hafta2` paketidir. Bu klasörü olduğu gibi ya da içindeki `.java` dosyalarını arşivleyebilirsiniz.

> **Hocanın vurgusu:** Gönderdiğiniz dosyanın gerçekten kod dosyası olduğundan emin olun. `build` altındaki `.class` dosyaları değerlendirilemez. Doğru olan `src` altındaki `.java` dosyalarını ya da export edilmiş projeyi göndermektir.

**Soru:** Hocanın paylaştığı kodlar bu yöntemle import edilemedi. Aynı yöntemle mi yapılacak?

**Cevap:** Hayır. Hoca o klasörleri export etmedi. NetBeans projeleri klasöründeki proje klasörlerini olduğu gibi kopyalayıp paylaştı. Böylece `src` klasörünü doğrudan bulmak kolaylaşıyor.

## 2. Bu Dersin Diğer Derslerle Bağlantısı

Hoca, IDE'nin kod üretme özelliklerini hatırlattı: Sağ tıklayıp `Insert Code` dediğinizde constructor, getter ve setter hazır oluşur, aynı özellik IntelliJ'de de vardır. Bunlar hızlı kod yazmayı sağlar. Önemli olan neyi nerede yapmanız gerektiğini bilmektir.

Ardından dersin ileriye dönük yerini anlattı. Dönemin sonunda tasarım kalıpları (design patterns) ve grafik arayüz gibi konular gelecek. Temel şimdi iyi atılırsa oradaki konular sorun çıkarmaz. Hocaya göre bu ders birkaç dersin kesişim noktasındadır: Veritabanı, yazılım mühendisliği ve sistem analizi ile tasarımı birleştiğinde "yazılım uzmanı" dediğimiz yetkinlik oluşur. Web ve mobil programlama da buradan geçer.

## 3. Kopya Yapıcı (Copy Constructor)

### 3.1 Bir Nesneyi Klonlamak

Geçen derste `toString` konusuna kadar gelinmiş, kopya yapıcı anlatılmadan kalmıştı. Hoca önceki derslerin `Kitap` ve `Yazar` sınıfları üzerinden devam etti. Slaytlardaki araba örneğinde de aynı mantığın geçerli olduğunu söyledi.

Bir kitap oluştururken yapıcıya (constructor) şimdiye kadar değer parametreleri veriyorduk. Yapıcıyı bir kez daha aşırı yükleyerek (constructor overloading) bir kitabı başka bir kitaptan klonlayabiliriz. İstediğimiz kullanım şudur:

```
Kitap k5 = new Kitap(k);
```

Yapıcı tanımlanmadan bu satır hata verir. Bu yapıcının diğerlerinden farkı şudur: Aldığı parametre **kendi türünden bir nesnedir**. Yani bir nesnenin kopyasını almak gibi çalışır. Bu tanım bütün sınıflarda yapılabilir.

> **Ders sonrası not:** Altyazıda "değerleme zamanı hatası" geçiyor, kastedilen derleme zamanı hatasıdır. javac ile denendi: Kopya yapıcı yokken bu satır `no suitable constructor found for Kitap(Kitap)` hatası verir.
>
> **Ders sonrası not:** Hoca "copy constructor Java'da olan bir kavram değil, C++'ta var" dedi. Dersin kendi kodu bunu sınırlar: `Kitap(Kitap k)` javac ile derlendi ve çalıştı. Yani Java'da elle yazılan bir kopya yapıcı vardır. Fark şudur: `Kitap` sınıfında bu yapıcı yazılmadığında javac hata verdi, Java kendiliğinden üretmedi. C++'ta ise kopya yapıcı yazılmayan bir yapıda `P b(a);` satırı `g++` ile derlendi ve çalıştı (`g++` ile denendi).

### 3.2 Yapıcının Yazılması

`Kitap` sınıfında zaten iki yapıcı vardı: `ID` ve `ad` alan yapıcı ile `ID`, `ad` ve `yazar` alan yapıcı. İkincisi, `ID` ve `ad` değerlerini `this(...)` ile birincisine gönderiyordu. Üçüncü olarak kitap türünden bir nesne alan yapıcı eklendi.

Gelen kitabın yazarı olabilir de olmayabilir de. Hoca bu nedenle önce `this(k.ID, k.ad)` ile iki parametreli yapıcıyı çağırdı. Sonra `this.yazar = k.yazar` yazdı. Burada `null` kontrolüne gerek yoktur. Nesne oluşturulurken bir şey atanmamış referans tipli alanlar `null` değerini alır. Eski nesnede yazar atanmışsa onun referansı, atanmamışsa `null` yeni nesneye geçer.

```
public Kitap(Kitap k) {
    this(k.ID, k.ad);
    this.yazar = k.yazar;
}
```

**Soru:** Buradaki `this(...)` tam olarak ne yapıyor?

**Cevap:** `this(...)`, bu nesnenin verilen parametrelere uyan yapıcısını çağırır. Burada bir `int` ve bir `String` alan yapıcı çağrılır. Üç parametreli yapıcıda parametreler dışarıdan gelir, kopya yapıcıda ise değerler var olan bir nesneden alınır. Değerler aynı yapıcıya gönderilir ve aynı atama kodu bir daha yazılmaz.

### 3.3 Birinci Deneme: İlkel ve String Alanlar

Değişiklikleri denemek için `Kitap` sınıfına set metotları gerekiyordu, çünkü alanlar `private`. Hoca bunları `Insert Code` ile ekledi. `Encapsulate Fields` ile yapınca `static` tanımlı `id` alanı için de `static` bir metot üretildi ve hata alındı. Hoca işlemi `Ctrl+Z` ile geri aldı ve `Insert Code > Setter` ile yalnızca gereken alanı seçti.

Deneme şöyle kuruldu: `k` kitabının `ID` değeri 1, adı `Java 1`'dir ve yazarı yoktur. `k5` onun kopyasıdır. Önce `k5` tanıtılır, sonra `k5` üzerinde `setAd("Java 5")`, `setID(5)` ve `setYazar(y2)` çağrılır. Soru şudur: Bu değişiklikler orijinal `k` nesnesini etkiler mi?

Sonuçta `k` değişmedi: Yine `Java 1`, yine "Yazar bulunamadı". `k5` ise `ID` 5, ad `Java 5` ve `y2` yazarıyla göründü. Orijinal nesne bozulmamış oldu. Hoca kopyalama sonrasındaki bu değişiklikleri zaten bunu görmek için yaptığını söyledi.

### 3.4 Referans Alanın Kopyalanması

**Soru:** Sınıflar Java'da referans tipi. `yazar` da bir sınıf. Yapıcıda aslında yazarın referansını almış olmuyor muyuz? Yani kopya yapmıyoruz. Bu durumda `k2`'nin yazarını değiştirirsek `k5` de değişir mi?

**Cevap:** Hoca önce sorunun `k2.setYazar(...)` kısmını denedi. `Yazar y3` oluşturuldu. `k5`, `k2`'den kopyalandı. Sonra `k5.setYazar(y2)` ve `k2.setYazar(y3)` çağrıldı. Beklenti, ikisinin farklı yazarları göstermesiydi. Çalıştırınca da öyle oldu.

Bunun nedeni şudur. İlk başta `y1` nesnesinin referansı `k2`'ye bağlandı. Kopyalama ile `k5` de aynı `y1` nesnesini göstermeye başladı. Ancak `k5` yeni bir nesnedir. `k2`'den başka bir bellek adresinde, kendine ait bir yazar referansı alanı vardır. İki alan şu an yalnızca aynı noktayı göstermektedir. `k5.setYazar(y2)` yalnızca `k5`'in kendi alanına `y2`'nin referansını yazar. `k2.setYazar(y3)` de `k2`'nin alanına `y3`'ün referansını yazar. Bu atamalar birbirini etkilemez.

Hoca bunu ekranda okla çizerek gösterdi. Çizimi düz metinle şöyle yazabiliriz:

```
Kopyalamadan sonra:   k2.yazar ---> y1 <--- k5.yazar
setYazar'lardan sonra: k2.yazar ---> y3
                       k5.yazar ---> y2
                       (y1 nesnesi yerinde kalır, değişmez)
```

Yani kopyalamada yazarın altındaki bilgiler klonlanmaz. Yalnızca referans klonlanır.

> **Ders sonrası not:** Hoca, kopyalamada `String` için "string havuzundan yeni bir string oluşturuluyor ve kopyalanıyor", `ID` için de "direkt deep copy yapılıyor" dedi. javac ile denendi: Kopya yapıcıdan sonra `k2.ad == k5.ad` sonucu `true` çıkar, yani `this(k.ID, k.ad)` yeni bir `String` üretmez, aynı referansı kopyalar. `k5.ad` alanına yeni bir değer atandığında `k2.ad` yine `Java 2` kalır (denendi). `ID` ise ilkel türdür ve doğrudan değeri kopyalanır. Bu nedenle `Kitap(Kitap k)` yapıcısı sığ kopya (shallow copy) yapar: `k2.yazar == k5.yazar` sonucu da `true` çıkar (denendi).

### 3.5 Ortak Nesnenin Kendisini Değiştirmek

Soruyu soran öğrenci asıl merakını netleştirdi: `setYazar` ile referansı değiştirmek değil, `k2`'nin gösterdiği yazarın **kendisi** üzerinde değişiklik yapmak `k5`'i etkiler mi?

Bunun için önce yazara ulaşmak gerekir. Hoca iki yol söyledi: Alan `public` yapılır ya da `getYazar()` yazılır. `Kitap` sınıfına `getYazar`, `Yazar` sınıfına da ad için `setAd` eklendi. `k2` o sırada `y1`'i gösteriyordu. Çağrı şu oldu:

```
k2.getYazar().setAd("küçük Ali");
```

Bu kez iki kitapta da "küçük Ali" görüldü. Çünkü artık referans üzerinden **asıl nesneye** ulaşıp onun değerini değiştirdik. İki kitap aynı nesneyi gösterdiği sürece değişiklik ikisinde de görünür.

> **Ek bilgi (derste anlatılmadı):** İki kitabın aynı yazarı paylaşmaması için yazarın da kopyasını oluşturmak gerekir. Örneğin `Yazar` sınıfına bir `Yazar(Yazar y)` yapıcısı eklenip kopyada `new Yazar(k.yazar)` yazılır. Buna derin kopya (deep copy) denir.

javac ile denendi: Bu durumda kopyanın yazar adı değiştirildiğinde orijinal kitabın yazar adı değişmez ve == karşılaştırması `false` verir.

### 3.6 Tam Kod ve Çıktı

Aşağıdaki kod, derste adım adım değiştirilen sınıfların bir araya getirilmiş halidir. `Kitap` sınıfı kısa nottaki kareden alınan koda dayanır. Derste eklenen set ve get metotları üzerine yazıldı.

```
// Yazar.java
public class Yazar {
    private String ad;
    public Yazar(String ad) { this.ad = ad; }
    public void setAd(String ad) { this.ad = ad; }
    public void yazarTanit() { System.out.println("--->Yazar: " + ad); }
}
```

```
// Kitap.java
public class Kitap {
    private int ID;
    private String ad;
    private Yazar yazar;
    static int id = 0;

    public Kitap(int ID, String ad) {
        this.ID = ID;
        this.ad = ad;
    }

    public Kitap(int ID, String ad, Yazar yazar) {
        this(ID, ad);
        this.yazar = yazar;
    }

    // Kopya yapıcı
    public Kitap(Kitap k) {
        this(k.ID, k.ad);
        this.yazar = k.yazar;
    }

    public void setID(int ID) { this.ID = ID; }
    public void setAd(String ad) { this.ad = ad; }
    public void setYazar(Yazar yazar) { this.yazar = yazar; }
    public Yazar getYazar() { return yazar; }

    public void KitapTanit() {
        System.out.println("[ID:" + this.ID + "--->AD:" + this.ad);
        if (this.yazar != null) {
            this.yazar.yazarTanit();
        } else {
            System.out.println("--->Yazar bulunamadı");
        }
    }
}
```

```
// Test.java
public class Test {
    public static void main(String[] args) {
        Yazar y1 = new Yazar("Ali");
        Yazar y2 = new Yazar("Yazar 2");
        Yazar y3 = new Yazar("Yazar 3");
        Kitap k = new Kitap(1, "Java 1");
```

```
        Kitap k2 = new Kitap(2, "Java 2", y1);

        System.out.println("--- 1. deneme: yazarsız kitabın kopyası");
        Kitap k5 = new Kitap(k);
        k5.KitapTanit();
        k5.setAd("Java 5");
        k5.setID(5);
        k5.setYazar(y2);
        k.KitapTanit();
        k5.KitapTanit();

        System.out.println("--- 2. deneme: referansı değiştirmek");
        k5 = new Kitap(k2);
        k5.setYazar(y2);
        k2.setYazar(y3);
        k2.KitapTanit();
        k5.KitapTanit();

        System.out.println("--- 3. deneme: ortak nesneyi değiştirmek");
        k2.setYazar(y1);
        k5 = new Kitap(k2);
        k2.KitapTanit();
        k5.KitapTanit();
        k2.getYazar().setAd("küçük Ali");
        k2.KitapTanit();
        k5.KitapTanit();
    }
}
```

javac ile derlenip çalıştırıldı, çıktı şudur:

```
--- 1. deneme: yazarsız kitabın kopyası
[ID:1--->AD:Java 1
--->Yazar bulunamadı
[ID:1--->AD:Java 1
--->Yazar bulunamadı
[ID:5--->AD:Java 5
--->Yazar: Yazar 2
--- 2. deneme: referansı değiştirmek
[ID:2--->AD:Java 2
--->Yazar: Yazar 3
[ID:2--->AD:Java 2
--->Yazar: Yazar 2
--- 3. deneme: ortak nesneyi değiştirmek
[ID:2--->AD:Java 2
--->Yazar: Ali
[ID:2--->AD:Java 2
--->Yazar: Ali
[ID:2--->AD:Java 2
--->Yazar: küçük Ali
[ID:2--->AD:Java 2
--->Yazar: küçük Ali
```

## 4. C++'ta Kopya Yapıcı ve Yıkıcı

Hoca, kopya yapıcının C++'taki karşılığına da kısaca baktı. Amacı, farklı dillerle de biraz aşinalık kazandırmaktı. Bu kısım internetteki örnek sayfalar üzerinden, hazırlıksız olarak anlatıldı.

C++'ta da `class` yazılır, çok büyük bir fark yoktur. Java'daki `package` yerine `namespace` gelir. Her üye için ayrı ayrı `public` ve `private` yazmak yerine sınıf tanımında üyeler gruplanır: `public:` bloğu ve `private:` bloğu.

Hocanın baktığı `Line` sınıfında basit bir yapıcı ve bir kopya yapıcı vardı:

```
Line(const Line &obj);
```

Bu, bizim yazdığımız yapıcının aynısıdır. Tek fark nesneyi referansla almasıdır. C'de bir argümanı adresle alırken * kullanıyorduk. Burada referans almak için & işareti farklı bir amaçla kullanılır. Prototiplerden sonra gerçekleştirim (implementation) kısmı gelir. Örnekte kopya yapıcı kendi bellek alanını ayırır ve nesneyi oraya kopyalar.

Sınıftaki bir diğer parça yıkıcıdır (destructor). ~ işaretiyle tanımlanır ve nesne yok edilirken çalışır. Hoca bunu önceki derste geçen `finalize` ile ilişkilendirdi. C++'ta bellek alanını `delete` ile boşaltmak programcının işidir.

Kopya yapıcının çağrıldığı yer şu satırdır:

```
Line line2 = line1;
```

Hoca burada = işaretinin, kopya yapıcının içinde tanımladığınız işlemi doğrudan çalıştırdığını söyledi. `line2` oluşur ve `line1`'in değerlerine sahip olur. Asıl kopyalama mekanizması budur. Java'da ise = böyle bir iş yapmaz, kopya yapıcıyı `new Kitap(k)` diye açıkça çağırırız. Hoca, operatör aşırı yüklemesini ileride konuşacaklarını da söyledi.

> **Ders sonrası not:** Hocanın anlatımı = işlecinin tanımlanmasıyla karışıyor. g++ ile denendi: `Line line2 = line1;` satırı bir ilk değer vermedir ve kopya yapıcıyı çalıştırır. Nesne zaten varken yazılan `line3 = line1;` ise kopya yapıcıyı değil `operator=` işlecini çalıştırır. Ayrıca hoca yıkıcıya "deconstructor" dedi, kısa nottaki karşılığı "destructor" (yıkıcı) şeklindedir.

Hoca iki kısa örneğe daha baktı. Birinde `x` ve `y` alanları olan bir `Point` sınıfı vardı. Burada metot tanımları sınıfın içinde yazılabiliyordu ve yapıcı, kopya yapıcıyla birlikte daha açıklayıcı görünüyordu. Diğerinde C++'ın kendi başvuru sayfasında `A a2 = a1` biçimindeki çağrılar ve örtük tanımlanan (implicitly declared) kopya yapıcılar geçiyordu. Hoca bunların ayrıntısına girmedi. C++'ın zorluğunun referans mekanizmasında olduğunu söyledi: &, işaretçi ve * kol kola yürür ve birbirine karıştırılabilir.

**Soru:** Java'da işaretçi (pointer) kullanabiliyor muyuz?

**Cevap:** Java'da doğrudan pointer yoktur. C++'ta vardır. C#'ta ise `unsafe` bölgede kullanılabilir.

## 5. Senaryodan Sınıf ve Özellik Çıkarma: Üniversite Bilgi Sistemi

### 5.1 Gereksinimden Sınıfa

Hocaya göre bir proje alındığında, örneğin bir ihaleye girildiğinde, kurum size programın nasıl olacağını her zaman tek tek söylemez. Bir mevzuat, kanun ya da resmi doküman olabilir. Onu okuyup gereksinim analizini (requirements analysis) kendiniz çıkarırsınız. Biz burada işin kolay tarafındayız, iş kuralları hazır belirlenmiş kabul ediyoruz.

Hoca ekranda bir üniversite bilgi sistemi senaryosu açtı. Öğrencilere beş dakika verdi ve senaryodan hangi nesnelerin, hangi özelliklerin çıkarılabileceğini düşünmelerini istedi. Sonra tek tek sorarak tasarımı birlikte kurdu.

### 5.2 Üniversite, Fakülte ve Öğretim Üyesi

En büyük sınıftan başlandı: **Üniversite**. Bir öğrenci içinde dekanlar, öğretim üyeleri ve bölümler olduğunu söyledi. Hoca bunların ayrı birer sınıf olduğunu, önce üniversiteye ait özelliklere bakmak gerektiğini söyledi. Üniversitenin fakülteleri ve rektörü vardır. **Fakülte** de bir sınıftır ve altında **bölümler** bulunur. Hoca burada geçen dersteki dolap ve raf ilişkisini hatırlattı. Aynı yapı kütüphane ve raflarda olduğu gibi devam eder.

Fakülteleri yöneten için "dekanlar" denince hoca düzeltti: Bir fakültede bir dekan olur. Ardından asıl tasarım sorusunu sordu. Rektör ve dekan için ayrı sınıflar mı oluşturacağız? Cevap hayır. Bir **öğretim üyesi** sınıfı olur. Rektör de dekan da bu sınıftan birer nesnedir.

Bu tartışmadan iki sonuç çıktı. Bir üniversitede $n$ tane fakülte, bir fakültede $n$ tane bölüm olabileceği için fakülteler ve bölümler **dizi** olarak saklanır. Rektör ve dekan ise **tek bir nesne referansıdır**.

Bir başka öğrencinin "dışarıdan rektör geldiğinde yeni sınıf" önerisine hoca şöyle cevap verdi: Rektör diye bir sınıf yok. Öğretim üyesi bir insandır ve bu insan bir sınıftır. Rektörün içeriden ya da dışarıdan gelmesi önemli değildir, nesnenin referansı buraya bağlanır. Kitapta da böyleydi. Yazar soyut bir kalıptır, yazarın nereden geldiğine bakmayız.

**Soru:** Rektöre özel yetkiler, yani metotlar eklemek istersek o zaman sınıf yapmak mantıklı olur mu?

**Cevap:** Öğretim üyesinin altında dekan ve rektör gibi sınıflara ayrı metotlar tanımlanacaksa kalıtım ilişkisi devreye sokulabilir. Öğretim üyesi üst sınıf, rektör ve dekan alt sınıf olur. Ancak başka bir yol daha vardır: Yetkiyi ayrı bir sınıf yapmak.

### 5.3 Yetki Sınıfı

Öğretim üyesinin ID, ad ve soyad özelliklerine ek olarak bir de **yetkileri** olabilir. Birden fazla yetki olabileceği için bu da bir dizidir. **Yetki** sınıfında yetki ID'si ve yetki adı bulunur. Hocanın örneğinde 1 numaralı yetki rektör, 2 dekan, 3 bölüm başkanı, 4 personel daire başkanıdır. Bir kişinin yetkileri arasında hangisi varsa o kişi o görevi üstlenir.

Burada dikkat edilecek nokta, ilk bakışta bir özellik gibi görünen şeyin ayrı bir nesne ya da sınıf olarak düşünülebilmesidir.

### 5.4 Derslik ve Malzeme

Bir öğrenci her fakültenin altında derslikler olması gerektiğini önerdi. Hoca buna fakültedeki derslikler dizisini ekledi ve blokların da düşünülebileceğini söyledi: Kaç blok varsa altında derslikler vardır.

**Derslik** sınıfına birlikte şu özellikler eklendi: kapı numarası veya ID, ad (bazen etiket verilir), türü (laboratuvar, konferans salonu gibi) ve kapasite. Hoca bir özellik daha ekledi: Dersliğin ekstra malzemeleri. Bu da bir **malzeme dizisidir**. **Malzeme** sınıfında ID ve ad bulunur. Projeksiyon cihazı, beyaz tahta, kara tahta gibi malzemeler önce nesne olarak oluşturulur, sonra dersliğe eklenir. Yani kitaba yazar, rafa kitap eklemekle aynı mantıktır.

### 5.5 Öğrenci ve Bölüm

Bir öğrenci **Öğrenci** sınıfını önerdi: öğrenci numarası, bölüm numarası, ad, soyad ve danışman (supervisor) ID'si. Hoca neden ID dediğini sordu. Danışman bir kişidir. Adı, soyadı, yetkileri gibi birçok bilgisi vardır. Bu nedenle öğrencinin içinde onun **referansı** saklanır. Bölüm için de aynısı geçerlidir. Bölümün adı, başkanı, adresi gibi nitelikleri vardır.

Hoca tahtada kendi uydurduğu bir gösterim kullandı: Altı çizili olmayan özellikler `int`, `String`, `char` gibi değerlerdir. Altı çizili olanlar referanstır. Diziler de ayrıca işaretlendi.

**Bölüm** sınıfının ID'si, adı, başkanı ve bağlı olduğu fakülte bilgisi vardır. Hocanın örneği şöyledir: Bir bölüm ID'si Bilgisayar Mühendisliği'ni gösterir. Bu bölüm Elektrik-Elektronik Fakültesi'ndedir. Fakülte bilgileri fakülte nesnesinden alınır. Oradan dekana, öğretim üyesine bölüm bilgisi eklenirse dekanın hangi bölümde çalıştığına ulaşılır.

Bu, biraz önceki `k2.getYazar().setAd(...)` örneğinin genişletilmiş halidir. Nokta koyarak nesneden nesneye atlayıp istenen bilgiye ulaşabiliriz.

### 5.6 Katalog Ders ve Açılan Ders

Bir öğrenci katalog dersleri önerdi. Dersin kodu, adı, kredisi ve bölüm bilgisi olacaktı. Hoca önce bu sınıfları tersten adlandırdı, sonra kendisi düzeltti: Doğrusu şudur.

- **Katalog ders:** Bölümün ders kataloğundaki derstir. Hocanın örneğiyle Bologna sayfasında 100 ders vardır. Bu sınıfta kod, ad, kredi ve bölüm bulunur.

- **Açılan ders:** Katalogdaki derslerden o dönem açılanlardır. Örneğin 100 dersin 70'i açılır. Aynı dersin birden çok şubesi olabilir. Nesneye Yönelik Programlama'nın birinci, ikinci ve üçüncü gruplarının her birinin kendine ait bir ID'si vardır. Ayrıca katalogdaki dersle eşleştiren ders kodu, grup kodu ve öğretim elemanı bulunur.

Senaryoya göre öğretim elemanı ve gruplar dizi olarak da tanımlanabilir, çünkü bir dersi birden fazla kişi verebilir. Açılan derste tekrar bölüm yazmaya gerek yoktur. Bölüm bilgisi zaten katalog ders üzerinden gelir. Öğrencinin kayıt olduğu dersler de öğrencide **açılan ders türünden bir dizi** olarak tutulur.

Bölüm sınıfında öğrenci listesi olacak mı? Hocaya göre olmaz. Eklenebilir, ama öğrenciyi bölümden çağırmak daha az maliyetlidir. Öğrenciyi bölüme ve bölümü öğrenciye bağlayan iki taraflı ilişki bazen gereksizdir ve ekstra iş yükü çıkarır. Hoca, bu sistem bir veritabanı üzerinde çalışacağı için veritabanı mantığıyla düşünmek gerektiğini de söyledi ve UML'e benzeyen bir veritabanı şeması gösterdi.

### 5.7 Neden Kalıtım Değil?

Açılan ders, katalog dersten kalıtım almaz. Her grup açıldığında ayrı ID alanları verilir. Açılan dersin kendine ait bir ID'si ve katalogdan alacağı alanlar vardır. Burada alt-üst ilişkisi varmış gibi görünse de kalıtım kullanılmaz. Doğrusu, kitap-yazar örneğindeki **association** ilişkisidir. Hoca "kalıtımdan mümkün olduğunca kaçıyoruz" dedi.

**Soru:** En alttan bir `Person` sınıfı ile modellemeye başlasak, adı ve soyadı orada olsa, öğrenci de onu kalıtsa kötü bir tasarım mı olur?

**Cevap:** Öğrenci, öğretmen, memur gibi 3-4 sınıfınız varsa ve ortak alanlar çok azsa, yani yalnızca ad ve soyad ortaksa, onları sınıflarda ayrı ayrı yazmak yeterlidir. Hoca üç neden saydı:

- **Maliyet:** Kalıtımda yapıcılar birden fazla kez çağrılır, bu da gereksiz bir yüktür.
- **Güvenlik:** Hocanın yetkileri ileride ayrılacak. Bir şey `private` yapılmamışsa, öğrenci "hocaya git, hocadan derse git, dersten not girmeye git" gibi nokta zinciriyle bir yol bulabilir.
- **Veritabanı entegrasyonu:** Yalnızca ad ve soyad için ekstra bir sınıf eklemek veritabanıyla entegrasyonda da sorun çıkarır.

Ortak olan şeyler çoksa, örneğin lisans ve yüksek lisans öğrencisi için, kalıtım kullanılabilir.

> **Ders sonrası not:** Hoca kalıtımda yapıcının "iki defa çağrıldığını" söyledi. javac ile denendi: Bir alt sınıf nesnesi oluşturulurken önce üst sınıfın, sonra alt sınıfın yapıcısı birer kez çalışır. Aynı yapıcı iki kez çalışmaz.
>
> **Ders sonrası not:** Hocanın güvenlik örneğindeki nokta zinciri kalıtıma özgü değildir. Derste kullanılan `k2.getYazar().setAd(...)` zincirinde kalıtım yoktur ve javac ile denendi: Bu zincir association üzerinden çalışır. Sorunun kaynağı büyük olasılıkla kalıtımdan çok, alanların ve metotların erişiminin yeterince kısıtlanmamasıdır.

## 6. Adres Defteri Örneği ve Yazılım Tasarımının Aşamaları

### 6.1 Aşamalar

Aradan sonra hoca bir adres defteri uygulaması sayfası gösterdi. Bu kaynak, kısa nota göre Gordon College'ın "Address Book Example" sayfasıdır. Sayfa bir projenin geçtiği aşamaları sırayla gösterir:

1. **Gereksinim analizi ve kullanıcı arayüzü:** Yazılımda neler olacağının hikayesi. Bunlara spesifikasyon (kısaca "spec") denir.
2. **Use case analizi:** Bir kullanıcı neler yapabilir? Bunlar aslında sınıflarda kullanacağımız metotlara ilişkin işlemlerin simgeleridir.
3. **Sequence diagram:** İşlemlerin daha algoritmik anlatımı. Şu nesne şuna gider, şundan cevap gelir. Önceki derslerde söz edilen istek (request) ve cevap (response) burada görülür.
4. **Tasarım:** Kim kiminle ilişkili olacak, adres defterine neler eklenecek?
5. **Sınıf diyagramı:** Yalnızca sınıf adları ve aralarındaki ilişkiler.

6. **Ayrıntılı sınıf tasarımı:** Gerçekleştirimden önce yapılır. Hangi alanların oluşacağı gösterilir.
7. **Kod:** Bazı sınıfların kodu ve çalıştırılabilir uygulama.

Mesaj şudur: Bu iş pat diye kod yazmakla olmaz. Hangi sınıflar olacak, hangi özellikler olacak? İlk başta özellik diye düşünülen şey sonradan sınıf olabilir mi? Önce bunlara bakmak gerekir.

### 6.2 Yetki Örneğinin Gerçek Sistemdeki Karşılığı

Hoca az önceki yetki sınıfını üniversitedeki sistemlerle örnekledi. Üniversitenin bilgi sistemine girdiğinizde sizin gördüğünüz ekranla hocanın gördüğü ekran farklıdır. Giriş yapıldıktan sonra yetki sorgulaması yapılır ve ekran buna göre gösterilir. Online Yıldız'a girerken de önce akademisyen mi, öğrenci mi olduğunuz sorulur. Bu da bir ön kontroldür. Sistem bu kontrolün yükünü istemciye yaymış olur. Hiçbir öğrenci akademisyen olarak giremez, hoca da öğrenci olarak giremez.

## 7. Nesneler Arası İlişkiler

Sınıflar belirlendikten sonraki iş, sınıflar arasındaki ilişkileri belirlemektir. Dersin bu kısmı ağırlıklı olarak İngilizce slaytlar üzerinden işlendi.

Şimdiye kadar nesne, sınıf, referans ve ilkel veri gördük. Bir nesneden başka bir nesneye mesaj göndermek için aralarındaki ilişkiyi belirlememiz gerekir. İlk olarak **ilişki türü** ve **ilişkinin kardinalitesi** belirlenir. Sık karşılaşılan ilişkiler şunlardır:

- Association
- Dependency
- Aggregation
- Composition
- Inheritance (kalıtım)

Bunların hepsi UML diyagramlarıyla gösterilebilir. UML'de farklı şekillerle çizilseler de genel adları ilişkidir (relation). Sınıflar arasında çizilseler de aslında sınıfların **örnekleri (nesneler) arasındaki ilişki** olarak okunmalıdırlar. Örnek olarak ders ile öğrenci, ders ile akademisyen, rektör ile üniversite, dekan ile fakülte verilebilir.

## 8. Association

Şimdiye kadar aslında hep bu ilişki türünü uyguladık. Örnek: Bir müşteri (`Customer`) ve onun siparişleri (`Order`). İkisi de ayrı nesnedir ve kendi sınıfları vardır.

- **Customer:** ID, ad, yaş, eğitim düzeyi, cinsiyet gibi özellikler.
- **Order:** ID, ürünler (ürün de ayrı bir nesnedir, burada ürün dizisi olur), sipariş tarihi, siparişin durumu (teslim edildi, reddedildi, iade edildi gibi).

Bir müşteri, örneğin Ahmet, çok sayıda sipariş verebilir. Bu mantıksal bağ association ile gösterilir. Diyagrama göre `Customer` sınıfında bir `Order` dizisi tanımlamanız gerekir.

Hoca bunu önceki örneklere bağladı: Kitabın yazarı ya da raftaki kitaplar gibi düşünün. Bir siteye girip alışveriş yaptınız, ödemeyi yaptınız. İş bittikten sonra bir `Order` nesnesi oluşur ve müşterinin içindeki sipariş dizisine eklenir.

Association'ın gösterimi düz çizgi ve oktur. Araya ilişkinin adı yazılır, uçlara kardinalite dereceleri verilir: bire çok, bire beş, çoka bir, çoka çok gibi.

### 8.1 Veritabanındaki Benzer Gösterim

Hoca bu gösterimin veritabanında kullanılan bir gösterime benzediğini söyledi. Hoca bunu bir kuş ayağına benzetti. İşaretler şöyle okunur:

- Tek çizgi **bir** demektir.
- Kuş ayağı biçimli işaret **çok** demektir.
- Çizginin yanında yuvarlak varsa **sıfır** olabilir demektir. Yani "bir çizgi ve bir yuvarlak" sıfır veya bir anlamına gelir.

Örnek olarak bir öğretim üyesi ile fakülte dekanlığı ilişkisini verdi. Bir öğretim üyesi bir fakültenin dekanı olabilir ya da hiç olmayabilir. Bütün öğretim üyeleri dekan olmayacağı için sıfır eklenir. Şemada ayrıca sıfır veya çok ve bir veya çok gösterimleri de vardır. Yazılım veritabanı üzerinde çalışacağı için nesne yönelimli tasarım bununla uyumlu gitmelidir.

> **Ders sonrası not:** Hoca gösterimin adını hatırlayamadı ("ördek mi, kaz mı") ve altyazıda "crossfoot" geçiyor. Kısa notta da geçtiği gibi bu, büyük olasılıkla crow's foot (karga ayağı) gösterimidir.

### 8.2 İlişkinin Özü: Sahiplik

Association'ın özü **sahipliktir (ownership)**. Bu, "has" ilişkisidir. `Customer` has `Orders`. Raf kitaplara sahiptir, kütüphane çok sayıda kitaba sahiptir. Mesaj gönderebilen nesne, mesajın alıcısını **üye alan (member field)** olarak tutar.

Sınıfın içine sipariş nesnelerini çağıran metotlar yazarız. Örneğin "tüm siparişlerimi listele" metodu, dizideki her siparişin kendi tanıtım metodunu çağırır. Tıpkı `KitapTanit` içinde `yazarTanit` çağırmak gibi:

```
class Order {
    private int id;
    Order(int id) { this.id = id; }
    void introduceOrder() { System.out.println("Order " + id); }
}

class Customer {
    private Order[] orders;
    private int orderCount;

    Customer(int capacity) { orders = new Order[capacity]; }

    void addOrder(Order o) {
        orders[orderCount] = o;
        orderCount++;
    }

    void listAllMyOrders() {
        for (int i = 0; i < orderCount; i++) {
            orders[i].introduceOrder();
        }
    }
}
```

`orders[i].introduceOrder()` çağrısı bir **mesajdır**. Bu mesajla `Order` nesnesinden bir cevap alırız ve onun üyelerine, yani hangi ürünlerin alındığına, fiyatına, tarihine ve durumuna ulaşırız.

## 9. Ok Yönü ve Kardinalite

### 9.1 Ok Yönü

Association'ın yönü farklı durumlarda farklı anlam taşır.

- **Yön önemsiz:** A'dan B'ye ya da B'den A'ya association olabilir, yön ayrıca belirtilmemiştir.
- **A, B'ye sahip (A → B):** A'nın örneği B'nin örneğine mesaj gönderebilir. Kitap-yazar örneğinde olduğu gibi sınıfın içinde karşı tarafın bir örneği vardır.
- **Çift yönlü:** A'dan B'ye ve B'den A'ya mesaj gönderilebilir.

Çift yönlü için hoca derste konuşulan örneği hatırlattı. Bölümdeki öğrencileri listelemek için bölümün içine öğrenciler dizisi konur. Öğrencinin bölümünü öğrenmek için öğrencinin içine bölüm konur. İkisi de yanlış değildir. Hangisinin kullanılacağının belirli bir yolu yoktur. Problemi okuyup anlayıp ne gerektiriyorsa ona göre seçeriz.

> **Hocanın vurgusu:** Okun yönü çok önemlidir, kimin kime mesaj gönderebileceğini belirler. Ok çizilmemişse ya iki yönlü bir bağlantı vardır ya da mimar yönü henüz düşünmemiştir. Bu, ileride iyileştirilmek üzere başlangıç tasarımı olabilir.

### 9.2 Kardinalite

Kardinalite tamsayılarla ve yıldız karakteriyle belirtilir:

| Gösterim | Anlamı |
|---|---|
| `0..*` veya * | Sıfır veya daha çok |
| `1..*` | Bir veya daha çok |
| `1` | Yalnızca bir |
| `0..1` | Sıfır veya bir |
| `1..11` | Birden on bire kadar |

Örneğin A'dan B'ye `1..11` yazıyorsa A'nın içinde en fazla 11 elemanlık bir B dizisi saklanabilir. Sabit sayı statik bir durumu gösterir. Hoca genelde yıldızın daha çok kullanıldığını söyledi.

## 10. Az Ayrıntılı ve Çok Ayrıntılı Sınıf Diyagramı

Adres defteri örneğine dönüldü. Basit sınıf diyagramında yalnızca nesnelerin adları vardır, özellikleri yoktur. Ayrıntılı sınıf tasarımında ise Java, C++ veya C# gibi bir dile bağlı gerçekleştirim bilgileri yazılır: alanlar, türler, metotlar.

İki gösterimden de ilişkinin türü ve kardinalitesi anlaşılır. Hangisini kullanacağınız size kalmıştır. Örneğin sınıf diyagramında `AddressBookApplication` ve `FileSystem` sınıfları ile aralarındaki ilişki görünür. Ayrıntılı tasarımda ise `AddressBookApplication` sınıfının içindeki alanlar görülür.

## 11. Gizli Bilgi (Hidden Information)

İlişkiler tasarlanırken bazı bilgiler diyagramda açıkça yazılmaz, gizli kalır. Örneğin "müşteri sipariş verir" denildiğinde ayrıntı açıkça söylenmemiştir. Bu ilişkinin arkasındaki mantığı biz anlamalıyız: `Customer` sınıfında `Order` türünden bir dizi bulunmalıdır.

> **Hocanın vurgusu:** Bu çok önemlidir. Hoca, asistanlığından beri sınavlarda genellikle bunun sorulduğunu söyledi. Ayrıntılı bir tasarım verilir, sınıfların arasında bir ok çizilir. Bunu görünce `Order` türünden $n$ elemanlı bir dizi ya da liste olması gerektiğini, bunun üye alan olacağını ve `private` tanımlanacağını anlamalısınız. Statik ya da dinamik olması önemli değildir. `public` kullanmak nesne yönelimli mantığa aykırıdır.

## 12. Dependency

Slayttaki listede geçen bir başka ilişki dependency'dir. A, B'ye bağımlıdır (A depends on B). Association'a benzer.

Fark şudur. Association'da has ilişkisi nesnenin **özellikleri (attribute)** bölümünde tanımlanır. Dependency'de ise nesnenin referansı **metotlar** bölümünde, yani bir metodun içinde kullanılır. Yani A'nın örneği B'nin örneğine bir metodun gövdesinde mesaj gönderir. Mesajın alıcısı üye alan değildir.

**Soru:** `Customer` ile `Order` arasındaki ok çift yönlü olsaydı nasıl olurdu?

**Cevap:** Karşı tarafa da yıldızlı bir uç eklenir. Yani `Order` tarafına üye olarak (ikinci bölüme) bir `Customer` dizisi yazılır. Bu çift yönlü ilişkidir. Hoca, sistemi yöneten firmanın müşteri bilgilerine zaten sahip olduğunu, bu bilgiyi tekrar bağlamanın her zaman gerekmediğini söyledi. Bazı durumlarda ise gerekir.

Hoca bu farkın ince olduğunu özellikle belirtti. Bazı kitaplarda dependency ile association aynı ya da neredeyse aynı kabul edilir. Çünkü referansın metodun içinde ya da dışında kullanılması arasında çok büyük fark yoktur.

Dependency'nin özü şudur: B ya bir metodun **parametresidir** ya da sahiplik olmadan **geçici olarak** kullanılır. Hocanın tarifine göre iki kullanım şöyle yazılır:

```
class B {
    void mesaj() { System.out.println("B mesaji aldi"); }
}

class A {
    void method1(B b) {          // parametre olarak
        b.mesaj();
    }
    void method2() {             // geçici olarak
        B gecici = new B();
        gecici.mesaj();
    }
}
```

Hoca adres defteri tasarımından örnek gösterdi: `AddressBookController`, `AddressBook` sınıfını kullanıyor. Sınıf diyagramında bu ilişki dependency olarak verilmiş. Ayrıntılı tasarımda ise `AddressBook` hem üye olarak saklanıyor hem de metotlarda kullanılıyor. Yani aynı yerde hem association hem dependency bulunabiliyor.

## 13. Tek Yönlü Association: Person ve Car

### 13.1 Sınıf Tasarımı

Hoca bundan sonra tek yönlü association'ın gerçekleştirimine geçti. Kodları `NYP02` klasöründe paylaştığını söyledi. Slayttaki tasarım düz metinle şöyledir:

```
Person (0..1 ile Car'a bağlıdır, ok Person'dan Car'a)
  Alanlar : name: String
            car: Car
  Metotlar: Person(name: String)
            getName(): String
            getCar(): Car
            setCar(car: Car)
            introduceSelf(): String

Car
  Alan    : plate: String
  Metotlar: Car(plate: String)
            getPlate(): String
```

`introduceSelf` aslında `toString` metoduna benzer, yalnızca adı farklıdır. Bir kişi sıfır ya da bir arabaya sahiptir. İlişki tek yönlüdür: `Person` içinde `Car` referansı var, `Car` içinde `Person` yok.

Hoca ince bir noktaya dikkat çekti: `setCar` metodunda `Car` nesnesi aslında dependency gibi kullanılıyor. Ancak `car` alanı özellikler bölümünde olduğu için bu daha güçlü bir ilişkidir ve ona tek yönlü association diyoruz.

Slayttaki bir başka kural şudur: Alan modeli (domain model) sınıfları ile onu kullanan, `main` metodu bulunan sürücü (driver, test) programı farklı paketlerde durmalıdır. Hoca, `Car` ve `Person` sınıfları ile iki test programının indirilebildiğini söyledi.

### 13.2 Neden `0..1`?

Slaytta soru vardır: Association'da neden `0..1`? Bu, senaryonuzdan gelen bir sahiplik bilgisidir. Başka bir senaryoda başka bir kardinalite verilebilir. Bu örnekte `car` değişkeninin değeri `null`

ya da herhangi bir `Car` referansı olabilir. Dizi gerekmez. Yani her kişinin arabası olmak zorunda değildir.

### 13.3 Diyagramdaki Gizli Bilgiler

Bu tasarımda iki gizli bilgi vardır. Hoca önce slayttaki ifadeyi yanlış okuduğunu, ardından asıl gizli bilginin `Car` yapıcısıyla ilgili olduğunu söyledi.

İlk gizli bilgi `Car(String)` yapıcısındadır. Diyagramda yapıcıya bir `String` geldiği görünür, ama bu değerin nereye atanacağı yazmaz. Bu metin `plate` alanına atanmalıdır. Plaka numarası araba oluşturulurken verilir ve sonradan değiştirilemez. Hoca bunu daha önceki slaytlardaki şasi numarası örneğine benzetti.

İkinci gizli bilgi `Person` tarafındadır. `Person(String)` yapıcısı yalnızca adı alır. `car` referansı yapıcıda alınmaz. Bu yüzden nesne üretildiğinde `car` otomatik olarak `null` olur. Sonradan `setCar` ile atanır. Senaryo "arabayı da yapıcıda alın" deseydi onu da eklememiz gerekirdi.

### 13.4 Kod

`Person` sınıfının kodu kısa nottaki kareden, `Car` sınıfı ise slayttaki gizli bilgiden yazıldı. `Car` sınıfının gövdesi altyazıda okunmuyor, fakat hoca `getPlate` içinde `plate` alanının döndürüldüğünü söyledi.

```
// Car.java
public class Car {
    private String plate;

    public Car(String plate) {
        this.plate = plate;
    }
    public String getPlate() { return plate; }
}
```

```
// Person.java
public class Person {
    private String name;
    private Car car;

    public Person(String name) {
        this.name = name;
    }
    public String getName() { return name; }
    public Car getCar() { return car; }
    public void setCar(Car car) { this.car = car; }

    public String introduceSelf() {
        String intro;
        intro = "Hello, my name is " + getName();
        if (car != null)
            intro += " and I have a car with license plate "
                  + car.getPlate() + ".";
        return intro;
    }
}
```

Plaka için set metodu yoktur. Çünkü plaka değişmez.

## 14. `null` Kontrolü ve Sağlam Kod

### 14.1 `if` Neden Var?

Hocanın sorusu şuydu: `introduceSelf` içindeki `if (car != null)` kontrolünü neden yapıyoruz? Öğrenciler sohbet penceresinden ve sözlü olarak cevap verdi: `NullPointerException` almamak için. `car` `null` ise `car.getPlate()` çalışma zamanı hatası verir.

Hoca açıkladı: Referans boşsa, yani nesne oluşturulmadıysa, onun `getPlate` metodunu kullanamam. Yapıcıda atama yapmadık, bu yüzden burada kontrol mecburidir. Yapıcıda `null` atasaydık da yine kontrol etmemiz gerekirdi. Kontrol yapılmazsa program bir istisna (exception) fırlatır ve kapanır. Program kapanınca her şeyi baştan başlatırsınız ya da istisna yakalama (exception handling) yazarsınız, ona ileride gelinecek. Hocanın benzetmesiyle Java sizi burada korur. C kullanıcıyı korumaz, ne yaparsanız size aittir. Java her ayrıntıyı düşünür, belki bu yüzden biraz daha yavaştır.

Hoca ek bir örnek verdi: `Integer i` tanımlayıp aşağıda `i = i + 5` yazamazsınız. İlk değer atanması gerekir, yoksa "null pointer exception ya da initialization exception" alınır dedi.

> **Ders sonrası not:** Hocanın `Integer i` örneğinde iki ayrı durum karışıyor. javac ile denendi: Metot içindeki yerel `int i;` veya `Integer i;` için `i = i + 5` satırı **derleme hatası** verir: `variable i might not have been initialized`. Sınıf alanı olan `Integer i` varsayılan olarak `null` olur ve aynı satır çalışma zamanında `NullPointerException` fırlatır. Sınıf alanı olan `int j` ise `0` olur, `j = j + 5` sonucunda `5` çıkar. "initialization exception" adlı bir hata denemelerde görülmedi. Bu durumda hata istisna değil, derleyici iletisidir.

### 14.2 Metotların Çalışma Sırası

Slayttaki bir başka ayrıntı şudur: Bir sınıfa metot eklediğinizde hangi sırayla çalışacaklarının garantisi yoktur. Hiç çalışmayabilirler de. Önce `getName`, sonra `getCar`, sonra `setCar` çağrılacak diye bir şey yoktur. Tanım sırası önemli değildir. Yapıcıyı en sona bile yazabilirsiniz. Yalnızca geleneksel olarak yapıcılar metot bölümünün en üstüne yazılır.

İstisnalar yapıcı ve sonlandırıcıdır (finalizer). Nesne oluşturulduğunda yapıcı doğrudan çalışır. Nesne silindiğinde ise hocanın anlatımına göre sonlandırıcı, çağrılmadan doğrudan çalışır.

> **Ders sonrası not:** Hoca nesne silindiğinde sonlandırıcının doğrudan çalıştığını söyledi. JDK 25 ile denendi: `finalize()` yazan bir sınıfta javac `deprecated and marked for removal` uyarısı verdi. `f = null;` yazıldıktan sonra program bitene kadar `finalize` mesajı basılmadı. Yani sonlandırıcının çalışmasına güvenilemez.

Sonuç olarak biri `Person` oluşturup ona araba atamayabilir. Çünkü kardinalite `0..1`dir. Bu yüzden atama durumunu göz önünde bulundurmalıyız.

### 14.3 Plakaya Dışarıdan Ulaşmak

Slaytta başka bir soru vardır: Arabası olmayan kişinin plakası nasıl öğrenilir? `getCar()` bir `Car` referansı döndürür. Bu referansla `getPlate()` çağrılabilir:

```
aziz.getCar().getPlate();
```

Hoca sordu: Bu yasal mı, değil mi? Cevap: Yasaldır. `getCar` `public`'tir ve kişi nesnesinden erişilir. Bir `Car` referansı döndürür, `null` da olsa bir değer döner. Car nesnesinin içindeki `plate` alanına doğrudan erişemeyiz, `public` olan `getPlate` üzerinden, yani "arka kapıdan" erişiriz. Yazarken ve derlerken sorun yoktur. Sağlıklı bir plaka ancak önce `Car` nesnesi oluşturulup kişiye atandıysa gelir. Atanmadıysa program çalışırken patlar.

> **Hocanın vurgusu:** Sorumluluğumuz sağlam kod (solid code) yazmaktır. Sağlam kod hatasız ve hatalara dayanıklı koddur. Kişinin arabası olup olmadığını kontrol etmeli, plakaya ancak arabası varsa erişmeliyiz. Arabası yoksa alanın değeri `null`'dır. Bu, `Car` nesnesinin henüz atanmadığı anlamına gelir.

## 15. Başlatma Testi, Referans ve Heap

### 15.1 `null` ile Karşılaştırma

Slaytta nesnenin başlatılıp başlatılmadığının nasıl test edileceği gösterildi. Bir nesne başlatıldıysa `null` ile karşılaştırma `false`, `!=` ile karşılaştırma `true` verir. Başlatılmadıysa, yani değeri `null` ise, sonuçlar tersine döner.

```
Object o1 = new Object();
Object o2 = null;
System.out.println(o1 == null);   // false
System.out.println(o1 != null);   // true
System.out.println(o2 == null);   // true
System.out.println(o2 != null);   // false
```

Sonuçlar javac ile derlenip çalıştırılarak doğrulandı. Değişken adları yukarıdaki denemede seçildi, slaytta başka olabilir.

### 15.2 Nesne Oluşturma ve Bellek

Slayttaki ana programda önce yalnızca referans tanımlanır:

```
Person octay;
```

Bu satırdan sonra ortada canlı bir nesne yoktur, yalnızca referans vardır. Sonra `new` ile nesne oluşturulur:

```
octay = new Person("Oktay Sinanoğlu");
```

Burada önce `new` işleci çalışır ve yapıcı çağrılır. Sonra oluşan nesnenin referansı değişkene atanır. Böylece referans canlı nesneye bağlanır. Değişken adı ve değeri **stack** bölümünde durur. Bu değer, **heap** bölümündeki nesnenin adresidir.

### 15.3 Ana Program ve Çalıştırma

Hoca kodu NetBeans'e kopyaladı ve paket adını değiştirdi. Slayttaki `rover` nesnesini ilk başta görmeyip hata verir sanmıştı, sonra nesnenin oluşturulduğunu gördü. Ayrıca ikinci nesnede oluşturma ve atama aynı satırda yapılır. Bu nesnenin arabası yoktur. Program aşağıdaki gibidir (plaka değeri yer tutucudur):

```
// Main01.java
public class Main01 {
    public static void main(String[] args) {
        Person octay;
        octay = new Person("Oktay Sinanoğlu");
        Car rover = new Car("PLAKA");
        octay.setCar(rover);
        System.out.println(octay.introduceSelf());

        Person aziz = new Person("Aziz Sancar");
        System.out.println(aziz.introduceSelf());
    }
}
```

javac ile derlenip çalıştırıldı, çıktı şudur:

```
Hello, my name is Oktay Sinanoğlu and I have a car with license plate PLAKA.
Hello, my name is Aziz Sancar
```

Hoca sonra `if` kontrolünü pasif hale getirip kodu yeniden çalıştırdı. İkinci nesnede `NullPointerException` alındı. Aynı deneme javac ile derlenip çalıştırıldı, çıktı şudur:

```
Hello, my name is Oktay Sinanoğlu and I have a car with license plate PLAKA.
Exception in thread "main" java.lang.NullPointerException: Cannot invoke "Car.getPlate()" beca
```

Arabası olmayan kişi için `aziz.getCar().getPlate()` çağrısı da derlenir, çalışırken `NullPointerException` fırlatır (javac ile derlenip çalıştırıldı).

## 16. Sequence Diagram ile İzleme

Hoca `introduceSelf` çağrısının etkileşimini önce genel diyagramla, sonra `main` programı üzerinden gösterdi. Slayttaki diyagram önce "interaction diagram" diye anıldı, hoca sonra "sequence diagram" diye düzeltti. Hangi nesne için çağrıldığı önemli değildir. Akış şöyledir:

1. `main` metodu ya da herhangi bir test sınıfı, `Person` nesnesine `introduceSelf` isteği gönderir.
2. `Person` önce `getName` metodunu çalıştırır. Ad zaten bu nesnede saklı olduğu için bu kolaydır.
3. Plaka değerini de öğrenmek gerekir. Bunun için `Person`, `Car` nesnesine `getPlate` mesajı gönderir. Bu referans `null` olabilir.
4. Cevap `Person` nesnesine döner.
5. Sonuç `null` değilse ad ve plaka birleştirilerek cevap verilir.
6. Sonuç `null` ise yalnızca kişinin adı nihai cevap olarak test nesnesine gönderilir.

Bir sonraki slayt `Main01` programını baştan sona izler. Hocanın saydığı sıra şudur:

1. `Person` nesnesi oluşturma (istek ve cevap).
2. `Car` nesnesi oluşturma (istek ve cevap).
3. `setCar` çağrısı.
4. `introduceSelf` çağrısı. `System.out.println` çağrısı izlenmemiştir.
5. İkinci `Person` nesnesini oluşturma.
6. Bu nesne için `introduceSelf` çağrısı ve yalnızca adı içeren cevap.

## 17. Diyagramlardaki Gizli İlişkiler

Slaytta gizli bilginin iki durumu gösterilir. Her birinde kapalı gösterim üstte, açık gösterim alttadır. Üstteki gösterimde ilişkiler gizli, alttakilerde açıktır.

**Association durumu:** A ile B arasında association vardır. Burada B nesnelerinin A'nın üyesi olduğunu düşünmeliyiz. Aynı anlama gelen diğer gösterimde boş elmas (hollow, white diamond) kullanılır. Bu, A'nın özellik bölümünde bir dizi B nesnesi olduğu şeklinde anlaşılır. Hoca bu şeklin ileride göreceğimiz **aggregation** olduğunu, association'ın özel bir hali olduğunu söyledi. Burada parça-bütün ilişkisi değil, bir araya getirme ilişkisi vardır. A ile B arasında sıkı olmayan bir bağ bulunur. Hoca raf ve kitap örneğini hatırlattı: Rafta $n$ tane kitap olabilir.

**Dependency durumu:** A'dan B'ye dependency varsa, A'nın metot bölümünde bir `B` nesnesi olduğunu düşünmeliyiz. Örneğin bir `method1` metodu B nesnesi kullanır.

Slaytta bu açıklama, köşesi kıvrık bir kutu içinde verilmişti. Bu da bir UML gösterimidir: Diyagrama ek bilgi vermek istediğinizde not kutusu kullanılır.

## 18. Çift Yönlü Association

Kişi için arabayı, araba için de kişiyi saklıyorsak **çift yönlü (two-way) association** tanımlarız. Bu ilişkiye de kardinalite verilir. `Person` ile `Car` arasında iki taraf da sıfır veya bir olabilir. Bu durumda dizi eklemeye gerek yoktur, tek referans yeterlidir.

Bu tasarımla sınıflara yeni metotlar gelir. `Person` tarafında `getName`, `getCar`, `setCar` vardır. `Car` tarafına arabanın sahibini (`owner`) tutan `getOwner` ve `setOwner` eklenir. Bunlar karşı sınıfın nesnesiyle ilgilidir. Plakayla ilgili metotlar ise `String` alanıyla ilgilidir. Her iki sınıf da `main` programından kullanılabilir.

Çift yönlü association neden gerekir? Bir önceki tek yönlü örnekte arabanın sahibini bulamayız, çünkü `Car` içinde sahip bilgisi yoktur. Arabanın sahibini de bulmak istiyorsak `Car` sınıfına ek bir üye eklemeliyiz.

Hocanın örneği şudur: Araba kiralama yapıyorsunuz ve elinizde yalnızca araba nesnesi var. “Bu araba hangi firmanın?” sorusunu soracaksınız. Araba bilgisinden firmaya gitmeniz gerekir. Çift yönlü ilişki bunu sağlar.

> **Ders sonrası not:** Hoca, `Person` ve `Car` farklı sınıflar olduğu için aralarında döngü (loop) olmadığını söyledi. Kastettiği, bu tasarımın sorun çıkarmadığıdır. Referans düzeyinde ise döngü vardır. javac ile denendi: `p.getCar().getOwner().getCar().getPlate()` zinciri derlenir ve çalışır, yani kişiden arabaya, arabadan yine kişiye dönülebilir.
>
> **Ek bilgi (derste anlatılmadı):** Çift yönlü ilişkide iki referans ayrı ayrı atanır. javac ile denendi: Yalnızca `p.setCar(c)` çağrıldığında `c.getOwner()` hâlâ `null` döner. İki tarafın tutarlı kalması için programcı `setOwner` çağrısını da yapmalıdır. Hocanın iki yönlü ilişkiyi “ek iş yükü” diye anmasının bir yönü büyük olasılıkla budur.

## 19. Veritabanı Tarafından Bakış

Hoca, aynı ilişkinin veritabanında biraz farklı kurulduğunu söyledi. Bir kişinin çok arabası olabilir, bir araba bir kişiye aittir. Bu bire çok bir ilişkidir. Bir tarafı kişi, çok tarafı arabadır. Kişi ID’si arabanın tablosuna eklenir. Hangi arabanın kime ait olduğuna kolayca ulaşılır.

Çoka çok ilişkide durum değişir. Sahiplik olmak zorunda değildir, association’a başka anlamlar da yüklenebilir. Örneğin kiralamada bir kişi $n$ tane araba kiralayabilir, bir araba da $n$ tane kişiye kiralanabilir. İki taraf da çok tarafı olur. Bu durumda veritabanına **üçüncü bir tablo** eklenir. Java tarafında ise üçüncü bir sınıf eklemeye gerek yoktur. Bir sınıftan diğerine ve diğerinden ona bu özellik sağlanabilir.

> **Ek bilgi (derste anlatılmadı):** Çok tarafına eklenen kişi ID’si alanına veritabanı dilinde büyük olasılıkla yabancı anahtar (foreign key) denir.

## Dersin Toparlaması

Bu derste önce bir projeyi `.zip` olarak dışa aktarıp içe almayı ve kaynak kodla derlenmiş dosyaları ayırmayı gördük. Ardından kopya yapıcıyı yazdık. İlkel alanların değerinin, referans alanların ise yalnızca adresinin kopyalandığını denemelerle izledik. C++’taki kopya yapıcı ve yıkıcıya kısaca baktık. Sonra bir üniversite senaryosundan sınıfları, özellikleri ve referansları çıkardık. İlk bakışta alt-üst gibi görünen ilişkilerin çoğu durumda kalıtımla değil association ile kurulacağını konuştuk.

İkinci bölümde nesneler arası ilişkilere geçtik. Association’ın sahiplik, dependency’nin ise metot içinde geçici kullanım olduğunu; okun mesaj yönünü, kardinalitenin de sayıyı gösterdiğini öğrendik. UML’deki gizli bilgiyi okuyup koda çevirmeyi `Person` ve `Car` üzerinde uyguladık. `null` kontrolünün neden sağlam kodun parçası olduğunu çalıştırarak gördük. Hoca slaytların 77. sayfasında kaldı. Çift yönlü association’ın ayrıntıları ile aggregation ve composition konuları sonraki derse kaldı. Hocanın belirttiği gibi ilişkiler bundan sonra daha karmaşık hale gelecek ve tasarım tarafı da eklenecek.

**Not:** İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.